

COURSE REPORT

Assignment 1 – Task 2

Multi-Class Classification Using K-Nearest Neighbors (KNN)

CIFAR-10 Image Classification

Formal report documenting the implementation, experimentation, generated outputs, and interpretation of the supplied Google Colab notebook.

1. Introduction

This report presents the implementation and evaluation of a K-Nearest Neighbors (KNN) classifier for multi-class image classification using the CIFAR-10 dataset. The work is based on the supplied EXP1 Jupyter/Google Colab notebook and follows the requirements specified for Task 2 of Assignment 1. The objective is to classify images into ten semantic categories while implementing the KNN procedure and distance calculations directly rather than using a pre-built KNN model.

The notebook starts with the complete CIFAR-10 training and test sets, containing 50,000 training images and 10,000 test images. To make the from-scratch experimentation computationally manageable, the notebook converts the images to grayscale, reduces their spatial resolution from 32×32 to 16×16 , flattens each image into a 256-dimensional feature vector, normalizes pixel values to the interval $[0,1]$, and then constructs a balanced experimental subset containing 500 training and 100 testing images per class.

Five distance measures are compared: Euclidean, Manhattan, Minkowski with $p=3$, Cosine distance and Hamming distance. The experiment evaluates K values of 1, 3, 5, 7 and 9. The final model is selected using testing accuracy, after which a ten-class confusion matrix and class-wise precision and recall are calculated manually.

1.1 Task objective

The specific goal is therefore not simply to obtain the highest possible CIFAR-10 accuracy, but to investigate how a KNN classifier behaves when the representation, distance metric and neighborhood size are varied. The resulting report records the exact experimental design and the outputs produced by the notebook.

2. CIFAR-10 Dataset

CIFAR-10 is a ten-class image dataset used in the notebook for the multi-class classification task. Each observation is a 32×32 colour image with three RGB channels. The original dataset loaded by the notebook contains 60,000 images in total: 50,000 training images and 10,000 test images.

Property	Value
Dataset	CIFAR-10
Total images	60,000
Training images	50,000
Testing images	10,000
Original image size	$32 \times 32 \times 3$
Number of classes	10

Images per class in full dataset	6,000
Experimental training subset	5,000 (500 per class)
Experimental testing subset	1,000 (100 per class)

2.1 Class labels

Numeric label	Class
0	airplane
1	automobile
2	bird
3	cat
4	deer
5	dog
6	frog
7	horse
8	ship
9	truck

The class labels are represented by integers from 0 to 9 in the notebook and are mapped to human-readable names through the `class_names` list. The experimental subset is explicitly balanced, so every class contributes exactly 500 training images and 100 testing images.

3. Initial Dataset Inspection

The notebook first loads CIFAR-10 using `tensorflow.keras.datasets.cifar10`. The labels are flattened from their original shape into one-dimensional arrays. The resulting shapes printed by the notebook are (50000, 32, 32, 3) for the training images, (10000, 32, 32, 3) for the test images, (50000,) for the training labels and (10000,) for the test labels.

Before preprocessing, the notebook displays a collection of fifteen training images. This provides a visual confirmation that the dataset contains varied colour images representing the ten classes. The figure is retained below as part of the experimental evidence.

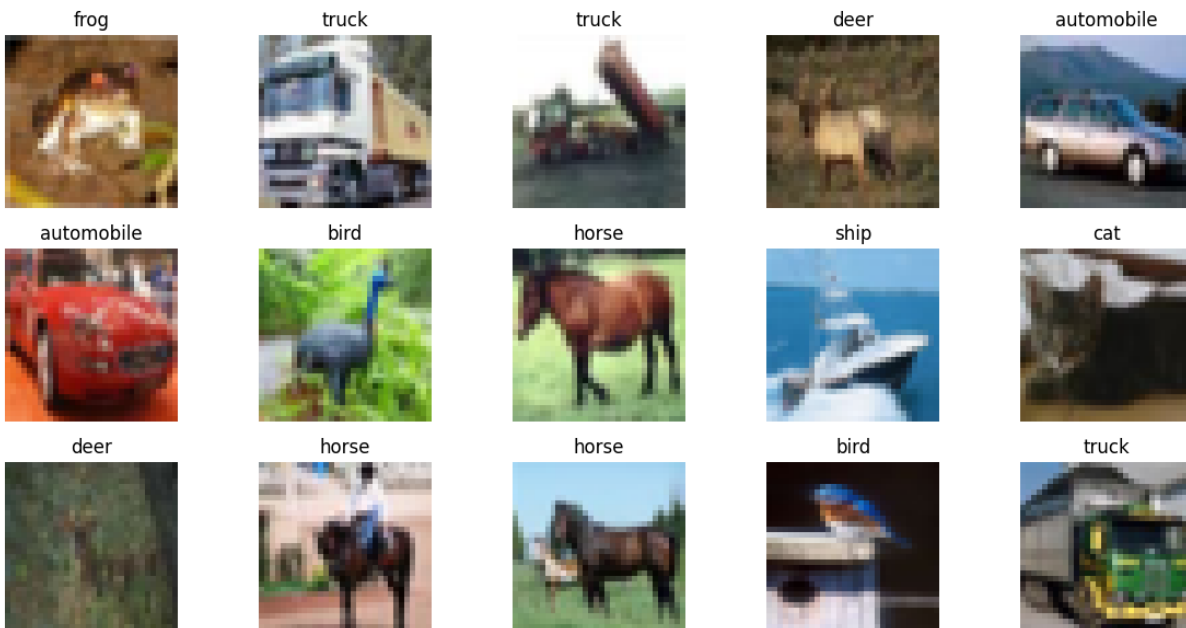


Figure 1. Fifteen CIFAR-10 training examples displayed by the supplied notebook.

3.1 Why image preprocessing is necessary

Directly applying a distance-based classifier to 32×32 RGB images would require comparing 3,072 values per image. The notebook therefore uses a simpler representation: grayscale conversion followed by spatial resizing. This reduces the number of input values to 256 per image and also makes the distance calculations substantially lighter.

4. Image Pre-processing

The preprocessing function processes each image individually. First, the RGB image is converted to grayscale using the weighted combination of the three colour channels: 0.299 for red, 0.587 for green and 0.114 for blue. The grayscale image is then resized from 32×32 to 16×16 using bilinear resampling.

The resulting images have shape (50000,16,16) for training and (10000,16,16) for testing. They are then flattened to one-dimensional vectors, giving 256 numerical features per image. Finally, each pixel value is divided by 255 so that the feature values lie between 0 and 1.

Stage	Training shape / representation	Purpose
Original	$50,000 \times 32 \times 32 \times 3$	Original colour images
Grayscale	$50,000 \times 16 \times 16$	Remove RGB channel dimension and reduce resolution
Flattening	$50,000 \times 256$	Create vectors for distance calculations
Normalization	$50,000 \times 256$ values in [0,1]	Put pixel magnitudes on a common scale
Experimental subset	$5,000 \times 256$	Balanced computational experiment

The notebook reports approximately 5.04 seconds for the grayscale and resizing preprocessing step. The subsequent flattened training and testing arrays have shapes (50000,256) and (10000,256), respectively.

5. Experimental Subset Construction

Although the complete CIFAR-10 data are loaded and preprocessed, the actual KNN experiments use a balanced subset. The notebook defines TRAIN_PER_CLASS = 500 and TEST_PER_CLASS = 100. For each of the ten classes, it selects the first 500 training examples and the first 100 testing examples encountered for that class.

The resulting experimental training set contains 5,000 observations and the experimental test set contains 1,000 observations. Because the same number of observations is selected from each class, the experimental evaluation is balanced at the class level. This is important for interpreting macro-averaged metrics and class-wise results.

Class	Training samples	Testing samples
airplane	500	100
automobile	500	100
bird	500	100
cat	500	100
deer	500	100
dog	500	100
frog	500	100
horse	500	100
ship	500	100
truck	500	100

The balanced design also means that a classifier with no useful discriminative ability would be expected to obtain accuracy near 10% under random guessing. The best observed accuracy in the notebook is 28.1%, which is substantially above that baseline but still indicates the difficulty of the image-classification problem under the simplified grayscale, low-resolution representation.

6. KNN Model Implemented from Scratch

The KNN implementation follows the standard instance-based classification procedure. For every test image, the notebook computes a distance between that image and every image in the experimental training set. The nearest K training observations are then selected using np.argpartition. Their class labels are counted using np.bincount, and the class with the largest count is returned as the prediction.

The implementation does not call a pre-built KNN estimator. Instead, the distance functions, prediction function, accuracy calculation, confusion matrix and precision/recall calculations are explicitly written in the notebook. This is consistent with the assignment requirement to avoid pre-built model functions.

6.1 Majority voting

For $K=1$, the prediction is simply the class of the nearest training example. For larger K values, the classifier considers a wider local neighborhood. The use of `np.argmax` on the class counts determines the final class label. The notebook therefore implements a conventional majority-vote KNN classifier for ten classes.

6.2 Computational procedure

- Select one test image.
- Calculate its distance to all 5,000 experimental training images.
- Identify the K nearest observations.
- Count the labels of those K neighbors.
- Assign the class with the largest count.
- Repeat for all 1,000 experimental test images.

7. Distance Metrics

Five distance measures are implemented independently. Their comparison is central to the experiment because KNN has no learned parameter vector that can compensate for an unsuitable distance function.

Metric	Implementation / interpretation
Euclidean	Square root of the sum of squared differences across the 256 pixel features.
Manhattan	Sum of absolute differences across the 256 features.
Minkowski	Generalized distance using $p=3$ in the supplied implementation.
Cosine	One minus cosine similarity; compares the orientation of feature vectors.
Hamming	Pixel vectors are thresholded at 0.5 and the mean proportion of positions that differ is calculated.

The Hamming implementation is different from the other measures because the continuous normalized pixel values are converted into binary values using a threshold of 0.5 before comparison. This means the metric effectively evaluates whether each pixel is below or above the threshold rather than using the original magnitude.

7.1 Choice of Minkowski parameter

The supplied Minkowski function uses $p=3$. The distance is therefore computed as the cube-root of the sum of the cubed absolute feature differences. This makes Minkowski a generalized alternative to Euclidean distance within the experiment.

8. Experimental Configuration

Parameter	Setting used in notebook
K values	1, 3, 5, 7, 9
Distance metrics	Euclidean, Manhattan, Minkowski, Cosine, Hamming
Minkowski p	3
Training subset	5,000 images
Testing subset	1,000 images
Features per image	256
Image representation	16×16 grayscale
Pixel scaling	0–255 divided by 255
Classes	10
Selection criterion	Highest testing accuracy

The experiment contains 25 separate model evaluations: five distance metrics multiplied by five K values. Each evaluation produces predictions for the same 1,000-image balanced test set, allowing the configurations to be compared directly.

8.1 Recorded execution time

The notebook also records the elapsed time for each KNN evaluation. Most configurations take approximately one second on the reported run, while Minkowski takes around 13 seconds per K value. Cosine at $K=7$ records 2.24 seconds. The

substantially higher Minkowski time is consistent with its more expensive element-wise exponentiation and root operations.

9. Accuracy Results

The following table reproduces the complete accuracy matrix produced by the notebook. Values are presented as percentages.

Distance	K=1	K=3	K=5	K=7	K=9
Euclidean	23.8%	22.6%	25.6%	26.7%	25.9%
Manhattan	24.8%	23.2%	25.8%	27.0%	28.1%
Minkowski	23.1%	20.5%	24.2%	22.6%	25.1%
Cosine	27.3%	24.4%	25.0%	27.1%	27.8%
Hamming	20.8%	22.4%	24.1%	25.0%	26.5%

The best individual result is 28.1%, obtained using Manhattan distance with K=9. Cosine distance gives the next strongest result at 27.8% with K=9, while Manhattan also records 27.0% at K=7. Euclidean reaches 26.7% at K=7, Hamming reaches 26.5% at K=9, and Minkowski reaches 25.1% at K=9.

9.1 Accuracy relative to random baseline

Because the experimental dataset contains ten equally represented classes, uniform random classification corresponds to an expected accuracy of approximately 10%. The selected KNN result of 28.1% is therefore 18.1 percentage points above this simple baseline. It is nevertheless much lower than the binary classification performance observed in the previous task, reflecting the substantially more difficult ten-class image problem.

10. Accuracy Visualization and Trend Analysis

The notebook generates a line plot showing how testing accuracy changes with K for each distance metric. This is one of the main required outputs of the assignment and provides a visual comparison of the five distance functions.

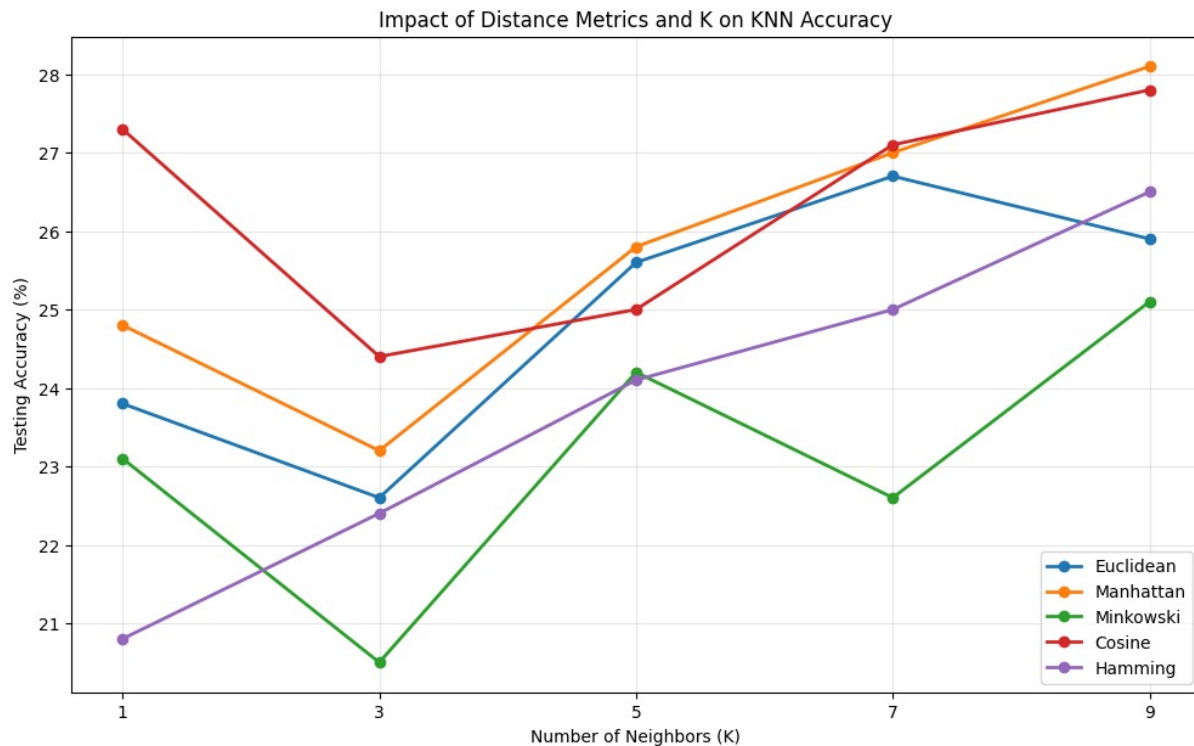


Figure 2. Impact of distance metrics and K on KNN accuracy, generated by the supplied notebook.

The plotted curves show that performance is generally highest at K=9 for Manhattan and Cosine distance. Euclidean improves from K=3 through K=7 before declining slightly at K=9. Hamming shows a generally increasing trend across the tested K values. Minkowski is less stable and remains below the other metrics for most K values.

10.1 Metric-wise averages

Metric	Mean accuracy	Best accuracy	Best K
Euclidean	24.92%	26.7%	7
Manhattan	25.78%	28.1%	9
Minkowski	23.10%	25.1%	9
Cosine	26.32%	27.8%	9
Hamming	23.76%	26.5%	9

11. Selection of the Best Model

The notebook searches all stored results and updates the best configuration whenever an accuracy larger than the current best is encountered. This procedure selects Manhattan distance with K=9 as the final configuration.

Selected characteristic	Result
Distance metric	Manhattan
K	9
Testing accuracy	28.1%
Correct predictions	281 / 1000
Incorrect predictions	719 / 1000

The selected model is therefore correct for 281 of the 1,000 experimental test images. The remaining 719 images are misclassified. This result is consistent with the confusion matrix and class-wise metrics presented in the following sections.

11.1 Why Manhattan is selected

Manhattan distance achieves 24.8%, 23.2%, 25.8%, 27.0% and 28.1% for K values 1, 3, 5, 7 and 9, respectively. Its highest result is the overall maximum across the 25 tested configurations. The increasing trend at the larger K values suggests that, for this representation, aggregating information from a wider neighborhood helps Manhattan distance more than it helps several of the alternative metrics.

12. Confusion Matrix Analysis

For the selected Manhattan K=9 configuration, the notebook calculates the ten-class confusion matrix from scratch. Rows represent actual classes and columns represent predicted classes.

Actual \ Pred	airplane	automobile	bird	cat	deer	dog	frog	horse	ship	truck
airplane	43	2	17	0	10	1	3	4	19	1
automobile	11	17	14	3	14	1	17	0	19	4
bird	22	3	48	4	10	2	7	3	1	0
cat	16	5	22	17	15	5	18	0	1	1
deer	21	5	25	2	24	1	7	3	11	1
dog	15	0	28	5	17	10	15	5	5	0
frog	13	1	31	7	19	0	25	1	3	0
horse	18	2	15	5	18	2	7	16	13	4
ship	12	4	5	0	14	3	1	1	59	1
truck	19	6	10	5	7	1	6	2	22	22

The diagonal entries represent correct classifications. The largest diagonal value is 59 for ship, followed by 48 for bird and 43 for airplane. The smallest diagonal value is 10 for dog. The matrix also shows substantial cross-class confusion, particularly among visually similar categories.

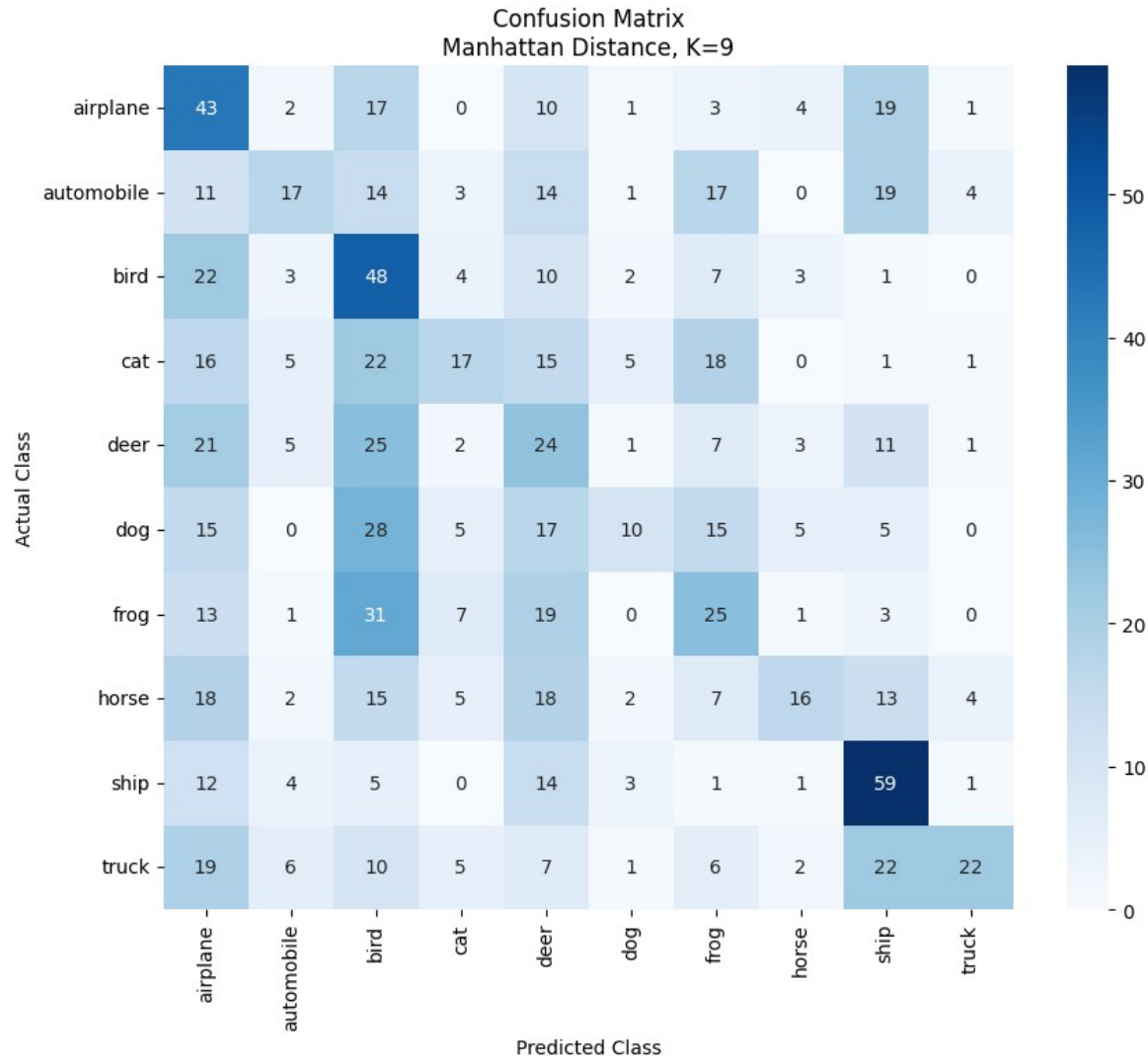


Figure 3. Confusion matrix heatmap for Manhattan distance with K=9, generated by the supplied notebook.

13. Precision and Recall

Precision and recall are calculated manually from the confusion matrix for each of the ten classes. Precision measures how often a prediction for a given class is correct, while recall measures how many actual examples of that class are successfully identified.

Class	Precision	Recall	Correct / 100
airplane	22.63%	43.00%	43
automobile	37.78%	17.00%	17
bird	22.33%	48.00%	48
cat	35.42%	17.00%	17
deer	16.22%	24.00%	24
dog	38.46%	10.00%	10
frog	23.58%	25.00%	25
horse	45.71%	16.00%	16
ship	38.56%	59.00%	59
truck	64.71%	22.00%	22

The highest precision is obtained for truck at 64.71%, meaning that when the model predicts truck, the prediction is correct relatively more often than for the other classes. The highest recall is obtained for ship at 59%, meaning that the model successfully identifies 59 of the 100 ship test images.

The lowest recall occurs for dog at 10%, with only 10 of the 100 dog images correctly classified. Deer has the lowest precision at 16.22%, indicating that many observations predicted as deer actually belong to other classes.

14. Macro-level Evaluation

Evaluation measure	Value
Testing accuracy	28.10%
Macro precision	0.3454 (34.54%)
Macro recall	0.2810 (28.10%)
Macro F1 derived from macro precision/recall	0.3099 (30.99%)
Correct predictions	281
Incorrect predictions	719

The macro precision and macro recall are both relatively low, reflecting the difficulty of obtaining consistently strong predictions across all ten classes. Macro recall equals the overall accuracy here because the experimental test set contains exactly 100 images for each class. In a balanced ten-class test set, the average of the ten class recalls is therefore equal to the total number of correct predictions divided by 1,000.

14.1 Stronger and weaker classes

Ship is the strongest class in recall terms, with 59 correct predictions. Bird follows with 48 and airplane with 43. Dog is the weakest class in recall, with only 10 correct predictions. The precision results show a different pattern: truck has the highest precision at 64.71%, while deer has the lowest at 16.22%. This distinction demonstrates why both precision and recall are useful in multiclass evaluation.

15. Class-wise Accuracy

The notebook calculates accuracy separately for each class by counting the correctly predicted examples among the 100 test observations belonging to that class. Since each class contains 100 test images, the class-wise accuracy values are numerically identical to the recall values reported in the previous section.

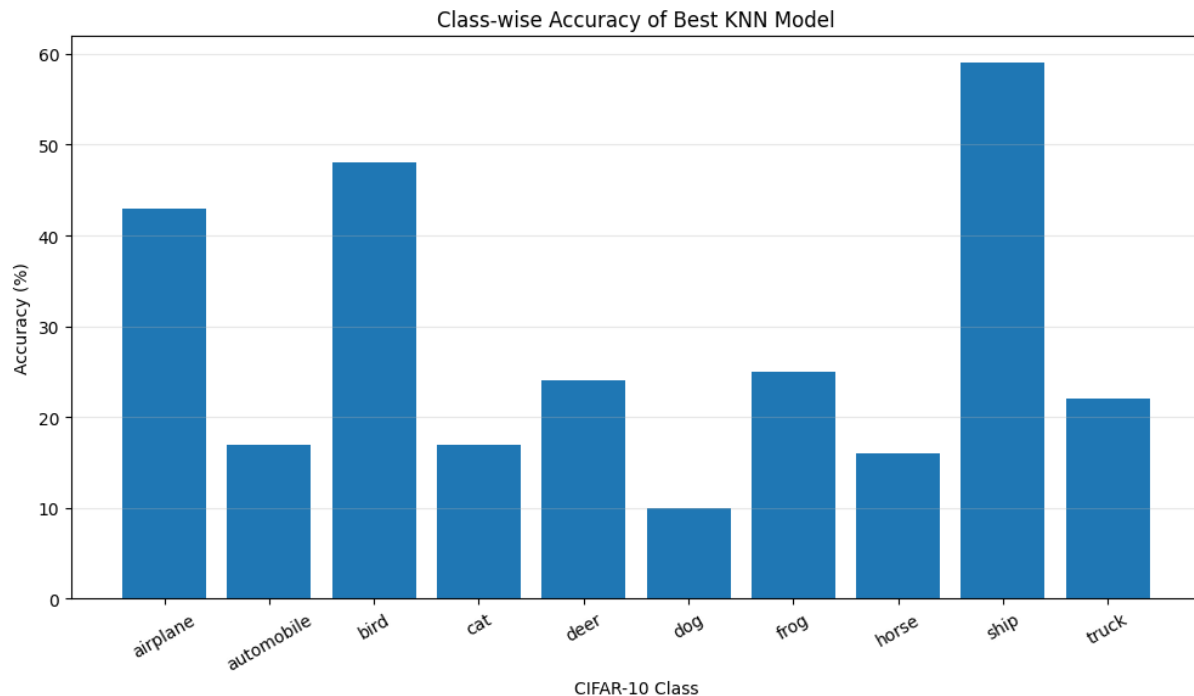


Figure 4. Class-wise accuracy of the best KNN model, generated by the supplied notebook.

The plot makes the uneven class performance visually clear. Ship has the highest class-wise accuracy at 59%, while dog has the lowest at 10%. The variation suggests that the low-resolution grayscale representation does not preserve enough discriminative information for some categories.

16. Visual Inspection of Misclassified Images

The notebook identifies the correctly and incorrectly classified test observations and reports 281 correct predictions and 719 wrong predictions. It then displays twelve examples selected from the misclassified set. For each image, the original CIFAR-10 colour image is displayed with its actual and predicted class labels.

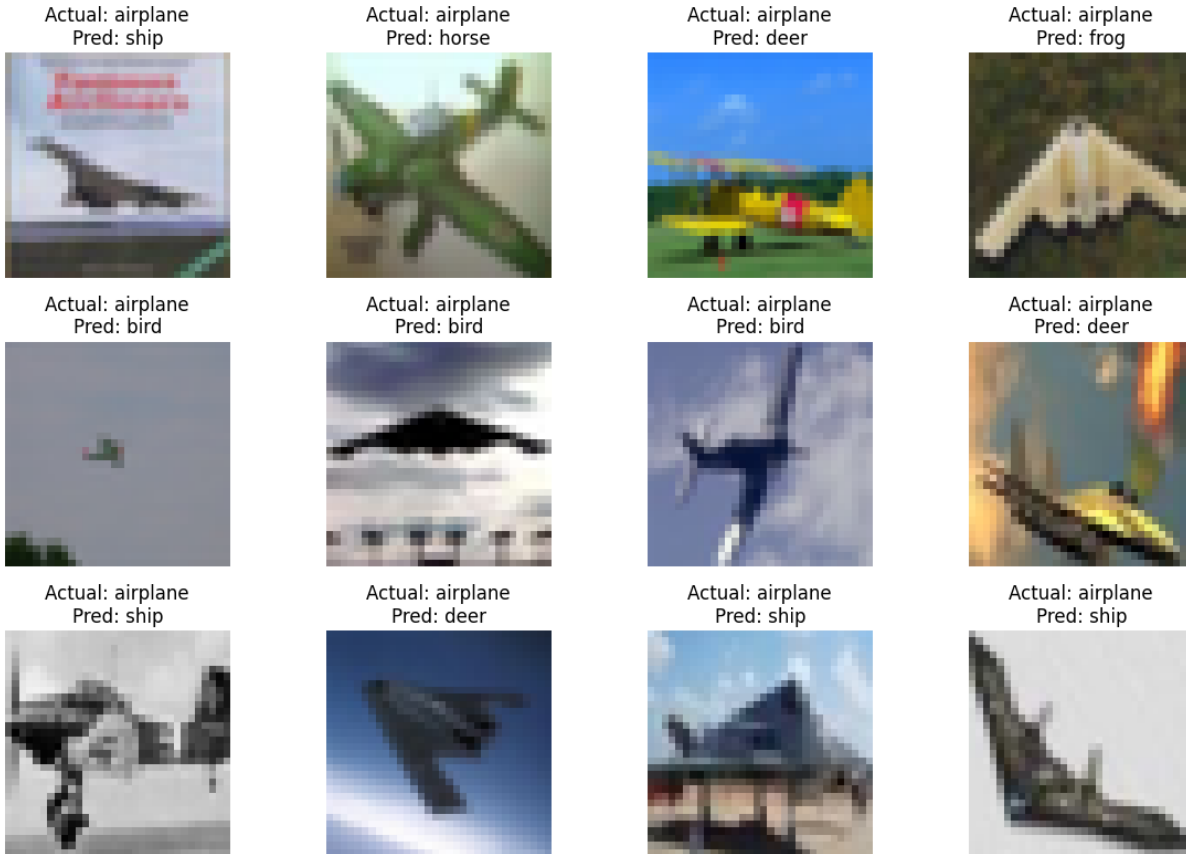


Figure 5. Twelve misclassified CIFAR-10 examples displayed by the supplied notebook.

These examples provide qualitative evidence for the numerical results. Many CIFAR-10 categories share similar shapes, textures and backgrounds. Once the images are converted to grayscale and reduced to 16×16 pixels, fine details that distinguish related classes can be lost. A distance-based classifier then has limited information with which to separate visually similar observations.

17. Computational Performance

The notebook records execution time for every metric and K combination. The timings demonstrate a significant difference between the distance implementations.

Metric	Average time (s)	Fastest (s)	Slowest (s)
Euclidean	1.01	0.88	1.19
Manhattan	1.10	1.00	1.27
Minkowski	12.92	12.85	13.09
Cosine	1.18	0.89	2.24
Hamming	1.00	0.98	1.03

Minkowski has an average execution time of approximately 12.92 seconds per K value, far above the approximately one-second range of most other metrics. The total Minkowski time across the five K values is approximately 64.61 seconds. By comparison, Manhattan takes approximately 5.48 seconds in total for the five K values.

These timing observations are relevant because KNN is computationally expensive at prediction time: every test observation must be compared with every training observation. The notebook reduces this cost by using only 5,000 training images rather than the full 50,000-image training set.

18. Discussion of Results

The overall results show that KNN can extract some class information from CIFAR-10 even after substantial dimensionality reduction, but its performance remains limited. The best accuracy of 28.1% is only moderately above the 10% balanced random baseline. The model therefore learns useful local similarity relationships, but those relationships are not strong enough to provide highly reliable ten-class classification.

Manhattan distance performs best in the supplied experiment. Its accuracy increases from 24.8% at $K=1$ to 28.1% at $K=9$, suggesting that local neighborhoods contain useful aggregate information for this representation. Cosine distance is also competitive, reaching 27.8% at $K=9$. Euclidean distance reaches 26.7%, while Hamming reaches 26.5%. Minkowski is consistently weaker and also considerably slower.

The confusion matrix indicates that errors are not uniformly distributed. Ship is recognized comparatively well, whereas dog is frequently confused with other classes. Bird, deer, frog, cat and dog share visual characteristics that become difficult to separate after grayscale conversion and resizing. The low recall for dog is particularly important because it shows that overall accuracy hides substantial variation between individual categories.

The experiment also illustrates the trade-off between computational simplicity and representational richness. Reducing each image from 3,072 colour values to 256 grayscale values makes a from-scratch KNN experiment feasible, but it removes colour and fine spatial information that can be important for CIFAR-10 recognition.

19. Key Observations and Inferences

- Manhattan distance with $K=9$ is the best configuration reported by the notebook, achieving 28.1% test accuracy.
- Cosine distance is the second strongest configuration at 27.8% for $K=9$.
- The best result is substantially above the 10% balanced random baseline, showing that the classifier captures some meaningful similarity between images.
- Performance improves with increasing K for Manhattan and Hamming in the tested range, whereas other metrics show less consistent trends.
- Minkowski distance is the most computationally expensive of the five tested metrics in the reported run.
- Ship has the strongest recall at 59%, while dog has the weakest recall at 10%.
- Truck has the highest class precision at 64.71%, while deer has the lowest at 16.22%.
- The confusion matrix shows extensive cross-class confusion, confirming that the model struggles to separate visually similar categories.
- The grayscale and 16×16 representation reduces computational cost but also removes colour and fine-grained spatial information.
- The balanced experimental subset makes macro-level comparisons straightforward because each class contributes equally to the test set.

Taken together, the results indicate that the distance function and K value materially affect KNN performance, but representation quality is also a major limiting factor. The experiment is therefore useful as an investigation of KNN mechanics rather than as a state-of-the-art CIFAR-10 classifier.

20. Limitations of the Experiment

- The full CIFAR-10 training set is reduced to 500 images per class for the actual KNN experiment, so only 5,000 of the 50,000 training images are used for prediction.
- The full-resolution RGB images are converted to 16×16 grayscale images, which removes colour information and fine spatial detail.
- The selected images are the first samples encountered for each class rather than a randomized sample; this is exactly how the `balanced_subset` function is implemented.
- The experiment uses a single balanced test subset of 1,000 images rather than the complete 10,000-image test set.
- No cross-validation is performed for choosing K .
- The KNN prediction procedure compares each test observation against all 5,000 experimental training observations, which limits scalability.

21. Conclusion

This report documented the complete KNN-based CIFAR-10 classification experiment contained in the supplied notebook. The implementation loads the complete CIFAR-10 dataset, converts images to grayscale, resizes them to 16×16, flattens them into 256-dimensional vectors, normalizes pixel values and constructs a balanced 5,000-image training subset and 1,000-image testing subset.

Five distance measures and five K values are evaluated without using a pre-built KNN model. The best configuration is Manhattan distance with K=9, producing 28.1% testing accuracy and 281 correct predictions out of 1,000. The macro precision is 0.3454 and macro recall is 0.2810. The confusion matrix and class-wise analysis show considerable variation between classes, with ship performing best by recall and dog performing weakest.

The experiment demonstrates that KNN can perform multiclass classification directly from image vectors, but the quality of the representation strongly influences the result. The reduced grayscale representation makes the experiment computationally practical while sacrificing information that could help distinguish visually similar CIFAR-10 classes.

Appendix A. Complete Experimental Results

Distance	K	Accuracy	Time (s)
Euclidean	1	23.8%	1.19
Euclidean	3	22.6%	0.89
Euclidean	5	25.6%	1.19
Euclidean	7	26.7%	0.88
Euclidean	9	25.9%	0.89
Manhattan	1	24.8%	1.00
Manhattan	3	23.2%	1.02
Manhattan	5	25.8%	1.01
Manhattan	7	27.0%	1.18
Manhattan	9	28.1%	1.27
Minkowski	1	23.1%	12.86
Minkowski	3	20.5%	12.85
Minkowski	5	24.2%	13.09
Minkowski	7	22.6%	12.92
Minkowski	9	25.1%	12.89
Cosine	1	27.3%	0.89
Cosine	3	24.4%	0.93
Cosine	5	25.0%	0.91
Cosine	7	27.1%	2.24
Cosine	9	27.8%	0.92
Hamming	1	20.8%	1.03
Hamming	3	22.4%	0.98
Hamming	5	24.1%	0.98
Hamming	7	25.0%	1.00
Hamming	9	26.5%	0.99

Appendix B. Confusion Matrix Values

Actual / Predicted	airplane	automobile	bird	cat	deer	dog	frog	horse	ship	truck
airplane	43	2	17	0	10	1	3	4	19	1
automobile	11	17	14	3	14	1	17	0	19	4
bird	22	3	48	4	10	2	7	3	1	0
cat	16	5	22	17	15	5	18	0	1	1
deer	21	5	25	2	24	1	7	3	11	1
dog	15	0	28	5	17	10	15	5	5	0
frog	13	1	31	7	19	0	25	1	3	0
horse	18	2	15	5	18	2	7	16	13	4
ship	12	4	5	0	14	3	1	1	59	1
truck	19	6	10	5	7	1	6	2	22	22

Appendix C. Notebook Implementation Summary

The supplied notebook uses NumPy for numerical operations, Matplotlib for figures, Seaborn for the confusion-matrix heatmap, pandas for tabular result presentation, PIL for image resizing, time for execution-time measurement, and tensorflow.keras.datasets.cifar10 for loading the CIFAR-10 dataset.

Notebook component	Role
numpy	Distance calculations, reshaping, label operations and manual evaluation metrics.
matplotlib	Sample images, accuracy plot, misclassified examples and class-wise accuracy plot.
seaborn	Confusion-matrix heatmap.
pandas	Results and precision/recall tables.
PIL.Image	Grayscale image resizing to 16×16.
time	Measurement of preprocessing and KNN execution time.
tensorflow.keras.datasets.cifar10	Loading the CIFAR-10 dataset.

The notebook does not use a pre-built KNN model from sklearn or another machine-learning library. The model and evaluation procedures required for the experiment are implemented explicitly.

Appendix D. Final Selected Model at a Glance

Item	Final result
Dataset	CIFAR-10
Experimental train set	5,000 images
Experimental test set	1,000 images
Feature representation	16×16 grayscale, flattened to 256 values
Best distance	Manhattan
Best K	9
Accuracy	28.10%
Correct	281
Incorrect	719
Macro precision	0.3454
Macro recall	0.2810

End of Assignment 1 – Task 2 Report